

The Streamlit spiking-neuron playground

ar003 · 16 June 2026

► **Open the live playground:** localhost:8501. Start it with `uv run streamlit run experiments/playground.py` (details below).

Most of this lab runs *batch-style*: a tool command takes a fixed set of arguments, runs a simulation, and drops a directory of artifacts that an experiment then publishes. That is the right shape for a result you want to pin down and cite.

The Streamlit app is the opposite end of that spectrum: a live, interactive surface for *building intuition* before you commit to a run. It re-simulates a single integrate-and-fire neuron on every slider drag and redraws the voltage trace immediately.

What it does

A radio button at the top of the sidebar picks the model, and the app drives the matching neuron subcommand: the **same** CLI the experiments run:

- **LIF**, the leaky integrate-and-fire neuron (`neuron lif`), a hard threshold with a spike-and-reset rule. This is the model behind exp000.
- **EIF**, the exponential integrate-and-fire neuron (`neuron eif`), where an explicit exponential term replaces the hard threshold. This is the model behind exp001.

Every parameter is a slider in the sidebar. Most are shared between the two models:

- **Input**, tonic current I and membrane resistance R_m .
- **Membrane**, time constant τ_m , and the resting / reset potentials.
- **Simulation**, duration and the integration timestep Δt .

The spiking mechanism, by contrast, is model-specific: LIF exposes a single hard threshold V_{th} , while EIF exposes the soft threshold V_T , the slope factor Δ_T , and the peak cutoff V_{peak} . As you drag, the app shows the active model, the spike count, and the firing rate in Hz, flagged *spiking* or *silent*.

The LIF membrane equation it integrates is

$$\tau_m \frac{dV}{dt} = -(V - V_{rest}) + R_m I,$$

with a spike-and-reset rule whenever V crosses V_{th} ; EIF adds the $\Delta_T \exp((V - V_T)/\Delta_T)$ term and resets when V reaches V_{peak} .

Starting the server

Launch it through `uv` (never call `python` / `streamlit` directly):

```
uv run streamlit run experiments/playground.py
```

Streamlit serves on `http://localhost:8501` by default and opens a browser tab. To pick another port:

```
uv run streamlit run experiments/playground.py --server.port 3001
```

Stop it with Ctrl-C in the terminal.

How it fits the rest of the lab

The playground deliberately sits **outside** the tool $\rightarrow$ experiment contract: it writes no `config.json` / `output.json` / `manifest.json` of its own to the record, produces no committed

artifacts, and isn't bundled by any experiment runner. It's a thinking tool, not a reproducible run, but it never reimplements the science: each slider settle shells out to the tool CLI and reads the trace back from `temp/neuron/<model>/`, the same files the experiments consume. When a slider configuration produces something worth keeping, reproduce it as a pinned run with the tool instead:

```
uv run python tools/neuron/tool.py eif --current 2.5 --duration 100 --delta-t 2
```

That writes the artifacts an experiment can publish.